

# Data Guardian API 文档

## 规则管理 API 文档

### 规则上传模块

#### 下载规则模板

获取规则导入的Excel模板文件。

请求URL: http://127.0.0.1:8080/rule/upload/

请求方式: GET

响应格式: Excel文件

响应说明:

- 成功: 返回Excel模板文件
- 失败: 返回404状态码及错误信息

#### 上传规则文件

上传Excel规则文件并保存到数据库。

请求URL: http://127.0.0.1:8080/rule/upload/

请求方式: POST

请求参数:

| 参数名  | 类型   | 是否必须 | 说明               |
|------|------|------|------------------|
| file | File | 是    | Excel文件(.xlsx格式) |

请求格式: form-data

响应格式: JSON

响应参数:

| 参数名           | 类型      | 说明                |
|---------------|---------|-------------------|
| status        | string  | 状态 (成功为"success") |
| message       | string  | 返回消息              |
| total_records | integer | 总记录数              |
| success_count | integer | 成功导入数量            |
| error_count   | integer | 错误记录数量            |

| 参数名           | 类型    | 说明             |
|---------------|-------|----------------|
| error_details | array | 错误详情，包含行号和错误信息 |

响应示例：

```
1  {
2    "status": "success",
3    "message": "成功导入 5 条规则",
4    "total_records": 6,
5    "success_count": 5,
6    "error_count": 1,
7    "error_details": [
8      {
9        "行号": 3,
10       "错误": "必填字段不能为空"
11     }
12   ]
13 }
```

错误码说明：

- 400: 文件格式错误或缺少必要列
- 500: 服务器内部错误

## 规则查询模块

### 获取项目名称下拉选项

获取所有不重复的项目名称作为下拉选项。

请求URL: http://127.0.0.1:8080/rule/search/project

请求方式: GET

示例URL：

```
1 | http://127.0.0.1:8080/rule/search/project
```

响应格式: JSON

响应参数：

| 参数名     | 类型    | 说明     |
|---------|-------|--------|
| options | array | 项目名称列表 |

响应示例：

```
1 | {
2 |   "options": ["项目A", "项目B", "项目C"]
3 | }
```

## 获取表格名称下拉选项

获取所有不重复的表格名称作为下拉选项，可根据项目名称筛选。

请求URL: http://127.0.0.1:8080/rule/search/table

请求方式: GET

请求参数:

| 参数名          | 类型           | 是否必须 | 说明       |
|--------------|--------------|------|----------|
| project_name | List[string] | 否    | 项目名称筛选条件 |

响应格式: JSON

单个项目示例URL:

```
1 | http://127.0.0.1:8080/rule/search/table?project_name=测试项目1
```

多个项目示例URL:

```
1 | http://127.0.0.1:8080/rule/search/table?project_name=测试项目1&project_name=测试项目2
```

响应参数:

| 参数名     | 类型    | 说明     |
|---------|-------|--------|
| options | array | 表格名称列表 |

响应示例:

```
1 | {
2 |   "options": ["表格1", "表格2", "表格3"]
3 | }
```

## 获取特征名称下拉选项

获取所有不重复的特征名称作为下拉选项，可根据项目名称和表格名称筛选。

请求URL: http://127.0.0.1:8080/rule/search/feature

请求方式: GET

请求参数:

| 参数名          | 类型           | 是否必须 | 说明       |
|--------------|--------------|------|----------|
| project_name | List[string] | 否    | 项目名称筛选条件 |
| table_name   | List[string] | 否    | 表格名称筛选条件 |

单个项目和表格示例URL：

```
1 | http://127.0.0.1:8080/rule/search/feature?project_name=测试项目1&table_name=表格A
```

多个项目和表格示例URL：

```
1 | http://127.0.0.1:8080/rule/search/feature?project_name=测试项目1&project_name=测试项目2&table_name=表格A&table_name=表格B
```

响应格式：JSON

响应参数：

| 参数名     | 类型    | 说明     |
|---------|-------|--------|
| options | array | 特征名称列表 |

响应示例：

```
1 | {
2 |   "options": ["特征1", "特征2", "特征3"]
3 | }
```

## 搜索规则数据

根据条件搜索规则数据，支持分页查询。

请求URL：http://127.0.0.1:8080/rule/search/

请求方式：GET

请求参数：

| 参数名          | 类型     | 是否必须 | 说明                  |
|--------------|--------|------|---------------------|
| project_name | string | 否    | 项目名称筛选条件            |
| table_name   | string | 否    | 表格名称筛选条件            |
| feature_name | string | 否    | 特征名称筛选条件            |
| start_time   | string | 否    | 创建开始时间 (YYYY-MM-DD) |
| end_time     | string | 否    | 创建结束时间 (YYYY-MM-DD) |

| 参数名       | 类型      | 是否必须 | 说明               |
|-----------|---------|------|------------------|
| page      | integer | 否    | 页码，默认为1          |
| page_size | integer | 否    | 每页数量，默认为10，最大100 |

基本搜索示例URL：

```
1 http://127.0.0.1:8080/rule/search/?project_name=测试项目1&table_name=表格A&page=1&page_size=20
```

多条件组合搜索示例URL：

```
1 http://127.0.0.1:8080/rule/search/?project_name=测试项目1&project_name=测试项目2&table_name=表格A&table_name=表格B&feature_name=特征X&start_time=2024-01-01&end_time=2024-03-24&page=2&page_size=15
```

响应格式： JSON

响应参数：

| 参数名   | 类型      | 说明     |
|-------|---------|--------|
| total | integer | 总记录数   |
| data  | array   | 规则数据列表 |

data数组中的对象结构：

| 参数名           | 类型          | 说明   |
|---------------|-------------|------|
| id            | integer     | 规则ID |
| project_name  | string      | 项目名称 |
| table_name    | string      | 表格名称 |
| feature_name  | string      | 特征名称 |
| rule_content  | string      | 对应规则 |
| error_type    | string      | 错误类型 |
| issue_details | string/null | 问题详情 |
| created_at    | string      | 创建时间 |

响应示例：

```
1 {
2   "total": 25,
```

```
3  "data": [  
4    {  
5      "id": 1,  
6      "project_name": "项目A",  
7      "table_name": "表格1",  
8      "feature_name": "特征1",  
9      "rule_content": "规则内容",  
10     "error_type": "类型错误",  
11     "issue_details": "详细问题描述",  
12     "created_at": "2023-03-15T12:34:56"  
13   },  
14   {  
15     "id": 2,  
16     "project_name": "项目B",  
17     "table_name": "表格2",  
18     "feature_name": "特征2",  
19     "rule_content": "规则内容2",  
20     "error_type": "逻辑错误",  
21     "issue_details": null,  
22     "created_at": "2023-03-14T10:24:36"  
23   }  
24 ]  
25 }
```

错误码说明：

- 400: 请求参数错误（如日期格式）
- 500: 服务器内部错误

重置搜索条件

重置所有搜索条件，返回所有规则数据（带分页）。

请求URL: http://127.0.0.1:8080/rule/search/reset

请求方式: GET

请求参数：

| 参数名       | 类型      | 是否必须 | 说明               |
|-----------|---------|------|------------------|
| page      | integer | 否    | 页码，默认为1          |
| page_size | integer | 否    | 每页数量，默认为10，最大100 |

示例URL：

```
1 | http://127.0.0.1:8080/rule/search/reset?page=1&page_size=20
```

响应格式: JSON

响应参数: 与搜索规则数据接口相同

响应示例：与搜索规则数据接口相同

错误码说明：

- 500: 服务器内部错误

## 规则管理模块

manageRuleRouter 的访问前缀是：`http://127.0.0.1:8080/rule/manage`

### 查看单条规则数据详情

获取指定 ID 的规则数据详细信息。

请求URL：`http://127.0.0.1:8080/rule/manage/{rule_id}`

请求方式：`GET`

路径参数：

| 参数名     | 类型      | 是否必须 | 说明           |
|---------|---------|------|--------------|
| rule_id | integer | 是    | 规则ID，必须大于等于1 |

响应格式：`JSON`

响应参数：

| 参数名           | 类型          | 说明   |
|---------------|-------------|------|
| id            | integer     | 规则ID |
| project_name  | string      | 项目名称 |
| table_name    | string      | 表格名称 |
| feature_name  | string      | 特征名称 |
| rule_content  | string      | 对应规则 |
| error_type    | string      | 错误类型 |
| issue_details | string/null | 问题详情 |
| created_at    | string      | 创建时间 |
| updated_at    | string      | 更新时间 |

响应示例：

```
1 {
2   "id": 1,
3   "project_name": "项目A",
4   "table_name": "表格1",
5   "feature_name": "特征1",
6   "rule_content": "规则内容",
7   "error_type": "类型错误",
8   "issue_details": "详细问题描述",
9   "created_at": "2023-03-15 12:34:56",
10  "updated_at": "2023-03-16 10:45:30"
11 }
```

错误码说明：

- 404: 指定ID的规则不存在
- 500: 服务器内部错误

更新规则数据

更新指定 ID 的规则数据。

请求URL: http://127.0.0.1:8080/rule/manage/{rule\_id}

请求方式: PUT

路径参数：

| 参数名     | 类型      | 是否必须 | 说明           |
|---------|---------|------|--------------|
| rule_id | integer | 是    | 规则ID，必须大于等于1 |

请求体：

```
1 {
2   "project_name": "更新后的项目名称",
3   "table_name": "更新后的表格名称",
4   "feature_name": "更新后的特征名称",
5   "rule_content": "更新后的规则内容",
6   "error_type": "更新后的错误类型",
7   "issue_details": "更新后的问题详情"
8 }
```

请求参数说明：

| 参数名          | 类型     | 是否必须 | 说明   |
|--------------|--------|------|------|
| project_name | string | 否    | 项目名称 |
| table_name   | string | 否    | 表格名称 |
| feature_name | string | 否    | 特征名称 |

| 参数名           | 类型     | 是否必须 | 说明   |
|---------------|--------|------|------|
| rule_content  | string | 否    | 对应规则 |
| error_type    | string | 否    | 错误类型 |
| issue_details | string | 否    | 问题详情 |

**注意：**所有字段均为可选，未提供的字段将保持原值不变。但至少需要提供一个字段进行更新。

**响应格式：** JSON

**响应参数：**

| 参数名     | 类型     | 说明                |
|---------|--------|-------------------|
| status  | string | 操作状态，成功为"success" |
| message | string | 操作结果信息            |

**响应示例：**

```
1 {
2   "status": "success",
3   "message": "成功更新ID为1的规则数据"
4 }
```

**错误码说明：**

- 400: 未提供任何需要更新的字段
- 404: 指定ID的规则不存在
- 500: 服务器内部错误

## 删除规则数据

删除指定 ID 的规则数据。

**请求URL：** http://127.0.0.1:8080/rule/manage/{rule\_id}

**请求方式：** DELETE

**路径参数：**

| 参数名     | 类型      | 是否必须 | 说明           |
|---------|---------|------|--------------|
| rule_id | integer | 是    | 规则ID，必须大于等于1 |

**响应格式：** JSON

**响应参数：**

| 参数名     | 类型     | 说明                |
|---------|--------|-------------------|
| status  | string | 操作状态，成功为"success" |
| message | string | 操作结果信息            |

响应示例：

```
1 {
2   "status": "success",
3   "message": "成功删除ID为1的规则数据"
4 }
```

错误码说明：

- 404: 指定ID的规则不存在
- 500: 服务器内部错误

## 批量删除规则数据

批量删除多条规则数据。

请求URL：`http://127.0.0.1:8080/rule/manage/batch-delete`

请求方式：`POST`

请求体：

```
1 {
2   "ids": [1, 2, 3, 4, 5]
3 }
```

请求参数说明：

| 参数名 | 类型    | 是否必须 | 说明         |
|-----|-------|------|------------|
| ids | array | 是    | 要删除的规则ID列表 |

响应格式：`JSON`

响应参数：

| 参数名     | 类型     | 说明                |
|---------|--------|-------------------|
| status  | string | 操作状态，成功为"success" |
| message | string | 操作结果信息            |

响应示例：

```
1 {
2   "status": "success",
3   "message": "成功删除5条规则数据"
4 }
```

错误码说明：

- 400: 删除ID列表不能为空
- 500: 服务器内部错误

# 大语言模型交互 API 文档

## LLM 对话模块

llmChatRouter 的访问前缀是：`http://127.0.0.1:8080/llm/chat`

### 与大模型对话(流式响应)

使用流式响应方式与大模型进行对话，适用于实时显示回答。

请求URL：`http://127.0.0.1:8080/llm/chat/stream`

请求方式：`POST`

请求体：

```
1 {
2   "query": "请问如何优化MySQL查询性能？",
3   "conversation_id": "conv_12345",
4   "user": "user_67890"
5 }
```

请求参数说明：

| 参数名             | 类型     | 是否必须 | 说明                           |
|-----------------|--------|------|------------------------------|
| query           | string | 是    | 用户的问题或输入                     |
| conversation_id | string | 否    | 对话ID，不提供则使用保存的值，提供空字符串则开始新对话 |
| user            | string | 是    | 用户标识符                        |

响应格式：`text/event-stream` (SSE)

响应示例：

```
1 data: {"event": "message", "answer": "要优化MySQL查询性能", "conversation_id":  
  "conv_abc123", "id": "msg_123456"}  
2  
3 data: {"event": "message", "answer": "要优化MySQL查询性能，您可以：", "conversation_id":  
  "conv_abc123", "id": "msg_123456"}  
4  
5 data: {"event": "message", "answer": "要优化MySQL查询性能，您可以：\n1. 建立合适的索引",  
  "conversation_id": "conv_abc123", "id": "msg_123456"}  
6  
7 ...  
8  
9 data: {"event": "message_end", "answer": "要优化MySQL查询性能，您可以：\n1. 建立合适的索引  
  \n2. 优化查询语句\n3. 合理设计表结构\n4. 使用查询缓存\n5. 适当分表分库", "conversation_id":  
  "conv_abc123", "id": "msg_123456"}  
10  
11 data: {"event": "done"}
```

错误响应：

```
1 data: {"event": "error", "message": "错误信息"}
```

与大模型对话(完整响应)

与大模型进行对话，返回完整的回答（非流式）。

请求URL: http://127.0.0.1:8080/11m/chat

请求方式: POST

请求体：

```
1 {  
2   "query": "请问如何优化MySQL查询性能？",  
3   "conversation_id": "conv_12345",  
4   "user": "user_67890"  
5 }
```

请求参数说明：

| 参数名             | 类型     | 是否必须 | 说明                           |
|-----------------|--------|------|------------------------------|
| query           | string | 是    | 用户的问题或输入                     |
| conversation_id | string | 否    | 对话ID，不提供则使用保存的值，提供空字符串则开始新对话 |
| user            | string | 是    | 用户标识符                        |

响应格式: JSON

响应参数：

| 参数名             | 类型      | 说明       |
|-----------------|---------|----------|
| answer          | string  | 大模型的回答内容 |
| conversation_id | string  | 对话ID     |
| created_at      | integer | 创建时间戳    |
| id              | string  | 消息ID     |

响应示例：

```
1 {
2   "answer": "要优化MySQL查询性能，您可以：\n1. 建立合适的索引\n2. 优化查询语句\n3. 合理设计表结构\n4. 使用查询缓存\n5. 适当分表分库",
3   "conversation_id": "conv_abc123",
4   "created_at": 1679123456,
5   "id": "msg_123456"
6 }
```

错误码说明：

- 500: 处理请求时发生错误
- 504: 与Dify通信超时

## 生成用户ID

生成一个唯一的用户ID，用于标识用户。

请求URL：`http://127.0.0.1:8080/11m/chat/generate-user-id`

请求方式：`GET`

响应格式：`JSON`

响应参数：

| 参数名     | 类型     | 说明        |
|---------|--------|-----------|
| user_id | string | 生成的唯一用户ID |

响应示例：

```
1 {
2   "user_id": "550e8400-e29b-41d4-a716-446655440000"
3 }
```

# 开始新对话

为指定用户开始一个新的对话。

请求URL: http://127.0.0.1:8080/11m/chat/new-conversation

请求方式: POST

请求体:

```
1 {
2   "user": "user_67890"
3 }
```

请求参数说明:

| 参数名  | 类型     | 是否必须 | 说明    |
|------|--------|------|-------|
| user | string | 是    | 用户标识符 |

响应格式: JSON

响应参数:

| 参数名     | 类型     | 说明     |
|---------|--------|--------|
| status  | string | 操作状态   |
| message | string | 操作结果信息 |
| user    | string | 用户标识符  |

响应示例:

```
1 {
2   "status": "success",
3   "message": "已开始新对话",
4   "user": "user_67890"
5 }
```

# 获取当前对话ID

获取指定用户当前的对话ID。

请求URL: http://127.0.0.1:8080/11m/chat/conversation-id/{user}

请求方式: GET

路径参数:

| 参数名  | 类型     | 是否必须 | 说明    |
|------|--------|------|-------|
| user | string | 是    | 用户标识符 |

响应格式: JSON

响应参数:

| 参数名             | 类型     | 说明                |
|-----------------|--------|-------------------|
| user            | string | 用户标识符             |
| conversation_id | string | 当前对话ID，如果没有则为空字符串 |

响应示例:

```
1 {
2   "user": "user_67890",
3   "conversation_id": "conv_abc123"
4 }
```